

RANK-IA: Sua organização de maneira eficiente

Trabalho apresentado como parte do processo de avaliação da disciplina de Engenharia de Software I do curso de Bacharelado em Ciência da computação da Faculdade de Ciências da UNESP de Bauru.

Integrantes do grupo:

RA	Nome (completo)
241025851	Fernando Hiroshi Murusaki
241025265	Igor dos Reis Gomes
231025335	Matheus Santos Magro
241024684	Murilo Tomaz Gonzaga

SUMÁRIO

1. Documento de especificação de requisitos.....	1
2. Modelagem dos requisitos.....	1
2.1. Introdução.....	1
2.2. Diagrama de Caso de Uso.....	1
2.2.1. Diagrama 1.....	2
2.2.2. Descrição do diagrama 1.....	2
2.3. Diagrama de Sequência.....	3
2.3.1. Visualização do Diagrama de Sequência.....	4
2.4. Diagrama de Atividade.....	4
2.4.1. Visualização do Diagrama de Atividades.....	5
2.5. Diagrama de Classes.....	5
2.5.1. Visualização do Diagrama de Classes.....	6
3. Processo de desenvolvimento de software.....	7
3.1. Introdução.....	7
3.1.1. Propósito.....	7
3.1.2. Objetivos.....	7
3.1.3. Justificativa do Modelo.....	7
3.2. Estruturação do processo Scrumban.....	8
3.2.1. Ferramenta de desenvolvimento.....	8
3.2.2. Definição das colunas do quadro.....	8
3.2.3. Ciclo de desenvolvimento.....	9
3.2.3.1. Duração.....	9
3.2.3.2. Incrementos entregáveis.....	9
3.2.3.3. Reavaliações contínuas.....	9
3.2.4. Papéis e Responsabilidades.....	10
3.2.4.1. Scrum Master.....	10

3.2.4.2. Product Owner.....	10
3.2.4.3. Developers.....	10
3.3. Planejamento das Sprints.....	10
3.3.1. Sprint 1.....	11
3.3.1.1. Visualização do quadro kanban.....	11
3.3.2. Sprint 2.....	12
3.3.2.1. Visualização do quadro Kanban.....	12
3.3.3. Sprint 3.....	13
3.3.3.1. Projeção da sprint.....	13
3.4. Execução e Acompanhamento.....	13
3.4.1. Daily meetings.....	13
3.4.2. Retrospectivas.....	14
3.4.3. Story Points.....	14
3.4.3.1. Story points no Taiga.....	14
4. Qualidade e métricas de software.....	16
4.1. Introdução.....	16
4.2. Objetivos da qualidade.....	16
4.3. Qualidade de Uso.....	16
4.4. Qualidade de Produto.....	17
4.4.1. Desempenho.....	17
4.4.2. Escalabilidade do software.....	18
4.4.3. Usabilidade.....	18
4.4.4. Segurança.....	19
4.4.5. Compatibilidade.....	19
4.4.6. Confiabilidade.....	19
4.4.7. Portabilidade.....	20
4.5. Métricas.....	20
4.5.1. Métricas de Uso.....	21
4.5.2. Métricas de Qualidade.....	21

1. Documento de especificação de requisitos

1.1. Introdução

1.1.1. Objetivo do documento

O documento em questão tem como objetivo descrever, de forma detalhada e estruturada, o escopo, descrição geral, requisitos funcionais, requisitos não funcionais, regras de negócio e os casos de uso do sistema proposto para formação inteligente de equipes.

É destinado aos analistas de sistemas, desenvolvedores, equipe de testes e stakeholders, em especial para o setor encarregado pelo gerenciamento de equipes, que serão os principais responsáveis pela gestão e operação do aplicativo.

Este documento então servirá como base para as etapas subsequentes do desenvolvimento, garantindo uma compreensão comum sobre o escopo e as funcionalidades esperadas do software

1.1.2. Escopo

O sistema tem como finalidade facilitar e otimizar o processo de montagem de equipes de trabalho dentro de uma organização, por meio de um aplicativo corporativo chamado RankIA.

O software utilizará um modelo de Inteligência Artificial(IA) que poderá ser treinado e capaz de compor equipes com base em informações como qualificações, competências e gaps individuais. Cada equipe gerada será acompanhada por um coeficiente de aproveitamento, que representa uma estimativa de produtividade esperada considerando o perfil coletivo dos integrantes

Além disso, o sistema permitirá ajustes manuais nas equipes

sugeridas, com objetivo de garantir a autonomia humana no projeto. Esses ajustes também servirão como feedback para o modelo de IA, que será aprimorado ao longo do tempo.

1.2. Descrição geral

1.2.1. Perspectiva do produto

O RankIA é um sistema corporativo voltado para a formação inteligente de equipes dentro de uma organização. O produto funcionará como um aplicativo web integrado aos processos internos do setor de Recursos Humanos, podendo também ser acessado pelos gestores e colaboradores.

O sistema será composto por alguns módulos principais, tais como:

- Cadastro e gerenciamento de colaboradores
- Mecanismo de recomendação baseado em Inteligência Artificial
- Interface de montagem e ajustes manuais de equipes
- Geração de relatórios e indicadores de produtividade
- Painel administrativo e de parametrização da IA

Com isso, o RankIA se posiciona como uma ferramenta híbrida, que combina automação inteligente com intervenção humana, buscando melhorar processos internos e aumentar a eficiência organizacional.

1.2.2. Funções do produto

O sistema terá como principais funcionalidades:

- Gerenciar e armazenar informações sobre colaboradores, como habilidades, competências, experiências e gaps.
- Gerar automaticamente equipes otimizadas com base nos dados cadastrados e nos critérios configurados.

- Exibir um coeficiente de aproveitamento para cada equipe sugerida, permitindo comparação entre diferentes formações.
- Permitir que o usuário ajuste manualmente as equipes sugeridas, removendo ou substituindo integrantes.
- Registrar os ajustes realizados e utilizá-los como feedback para o modelo de IA.
- Emitir relatórios sobre desempenho individual e coletivo dos colaboradores.
- Sugerir treinamentos de desenvolvimento profissional com base nos gaps identificados
- Disponibilizar visualizações gráficas das equipes formadas e de seus indicadores.
- Oferecer controle de acesso baseado em perfis.

1.2.3. Características dos usuários

Dentre os usuários do sistema, é possível dividir entre dois tipos de perfis principais:

- Profissionais de RH(Recursos Humanos) e ou Gestores:

Principais operadores do sistema. Irão realizar cadastros de colaboradores, configuração dos critérios da IA, validação e ajustes de equipes sugeridas e acompanhamento de relatórios. Além disso, serão os responsáveis por coletar as informações bases dos colaboradores desde sua entrada, assim como realizar análises das formações sugeridas e visualização dos indicadores de produtividade.

- Colaboradores:

Terão acesso limitado, podendo consultar suas competências registradas, acompanhar sua evolução, verificar os treinamentos sugeridos e visualizar como e quais são as equipes onde foram alocados.

Todos os usuários devem possuir um nível mínimo de familiaridade com

sistemas web corporativos para utilização correta do software.

1.2.4. Restrições

Dentre as condições e limitações impostas ao projeto que devem ser obrigatoriamente cumpridas, temos:

- O sistema deve operar como uma aplicação web, acessível via navegadores.
- A coleta e manipulação de dados de colaboradores devem seguir as normas de privacidade e segurança (LGPD).
- O modelo de IA deve operar somente sobre os dados fornecidos pela organização
- O sistema só poderá realizar ajustes automáticos mediante autorização definida nas regras de acesso
- Alterações com alto nível de importância como; exclusão de usuários, redefinição de dados da IA, trocas manuais de membros e afins só poderão ser feitas pelos administradores do sistema.

1.2.5. Suposições e dependências

Entre as suposições e dependências do projeto, como a disponibilidade de tecnologias ou infraestrutura, sistemas, softwares ou serviços externos dos quais o projeto necessita para funcionar corretamente, estão:

- Pressupõe-se que a organização manterá os dados dos colaboradores atualizados para garantir a precisão das equipes geradas.
- A efetividade das recomendações da IA depende da qualidade e consistência dos dados cadastrados.
- O funcionamento do sistema depende de infraestrutura de rede adequada e servidores capazes de suportar processamento de IA.
- Supõe-se que os usuários receberão treinamento inicial para utilização do sistema

- Depende de uma política de autenticação e autorização já existente ou desenvolvida junto ao sistema.

1.3. Requisitos

1.3.1. Requisitos funcionais

Para os requisitos funcionais do projeto, aqueles que tangem as funcionalidades específicas do sistema, há:

- **RF01:** O sistema deve permitir a troca manual de membros nas equipes sugeridas pela IA.
- **RF02:** O sistema deve ser capaz de coletar e armazenar informações relevantes sobre os jogadores, como habilidades, experiências, histórico de desempenho, e outros atributos.
- **RF03:** O software deve gerar automaticamente times balanceados com base nas classificações e atributos dos jogadores.
- **RF04:** O sistema deve ser capaz de gerar relatórios sobre o desempenho dos funcionários
- **RF05:** O sistema deve ser capaz de identificar os pontos fracos de cada funcionário e sugerir treinamentos relacionados à essas deficiências
- **RF06:** O software deve ser capaz de medir a qualidade das equipes e mostrar elas de forma gráfica

1.3.2. Requisitos não funcionais

Para os requisitos não funcionais, os que são atributos de qualidade do sistema, como segurança, desempenho, usabilidade e afins.

- **RNF01:** Senhas de usuários devem ser criptografadas.
Métrica: Deve ser realizado processo de criptografia na fase de desenvolvimento, com uma vertente forte.

- **RNF02:** A IA deve ser tunada com frequência de 1 semana.
Métrica: Para testes gerais do software, devem ser realizados tuning/treinamento da IA de forma frequente pelo time de QA
- **RNF03:** Envio de imagens, arquivos e anexos que devem apresentar segurança no front e no back.
Métrica: As rotas devem ser revisadas e configuradas por parte do time de desenvolvimento e testadas pela equipe de Pentest.
- **RNF04:** O software deve ter performance qualificada em sistemas operacionais diversos (Linux, Windows, MacOS).
Métrica: Testar e garantir que funcione em sistemas operacionais diferentes.
- **RNF05:** O software deve funcionar em uma margem grande de navegadores disponíveis no mercado(Firefox, Chrome, Edge...).
Métrica: O software vai ser WEB, portanto é preciso que utilizem ferramentas de desenvolvimento WEB que funcionam em navegadores Chrome based e em outros também.

1.4. Regras de negócio

Para as regras de negócio do projeto, aquelas que dirão respeito às diretrizes, políticas e restrições que irão definir as condições para a operação da empresa a utilizar e irão determinar o que é ou não permitido no contexto do projeto, temos:

- **RN01 - Atualização obrigatória de dados:**
Os colaboradores devem ter seus dados de competências, habilidades e experiências atualizados periodicamente pelo setor de RH, garantindo precisão nas recomendações da IA.
- **RN02 - Critérios mínimos para formação de equipes:**
Somente colaboradores com dados completos podem ser considerados na formação automatizada de equipes.
- **RN03 - Ajustes manuais como feedback obrigatório:**
Toda alteração manual realizada pelo RH ou gestor deve ser

registrada, contendo motivo e mudanças feitas. Esses dados alimentam o modelo de IA.

- **RN04 - Acesso baseado em permissões:**

Cada perfil de usuário tem acesso restrito a determinadas funcionalidades:

- RH ou Gestores: acesso total ao sistema.
- Colaboradores: acesso a informações pessoais e treinamentos sugeridos.

- **RN05 - Coeficiente de Aproveitamento(CA):**

Cada equipe formada deve gerar automaticamente seu coeficiente de aproveitamento, calculado com base em métricas definidas previamente

- **RN06 - Relatórios e Treinamentos sugeridos:**

O sistema deve gerar relatórios obrigatórios contendo métricas de desempenho das equipes e evolução individual dos colaboradores. A partir disso e dos gaps evidenciados o sistema deve sugerir os treinamentos específicos presentes no banco de dados e registrá-los no histórico do usuário.

- **RN07 - Versionamento de equipes:**

Cada equipe gerada deve ser registrada como uma versão, permitindo rastrear modificações feitas ao longo do tempo. Isso garante auditoria e transparência no processo.

1.5. Casos de uso e Histórias de usuário

1.5.1. Casos de uso

Os casos de uso apresentados a seguir oferecem uma visão narrativa clara e estruturada das interações realizadas pelos diferentes atores do sistema. Seu propósito é mapear e especificar o comportamento esperado do sistema ao longo do fluxo de execução de suas funcionalidades mais relevantes ou de maior

complexidade.

Nesta etapa do projeto, os casos de uso documentados compõem o núcleo inicial das funcionalidades planejadas. Outros casos de uso serão ampliados, revisados e adicionados progressivamente, acompanhando a evolução do processo de desenvolvimento e o refinamento contínuo dos requisitos.

ID	UC-01
Caso	Gerenciar colaboradores
Referência	"Formulário para cadastro de funcionários" e História #6: "Modelo de leitura de PDFs/currículos".
Atores	RH / Gestor de Equipes
Resumo	Este caso de uso permite que o Gestor cadastre novos colaboradores no sistema, seja preenchendo um formulário manualmente ou importando dados automaticamente a partir de um currículo em PDF. O objetivo é centralizar as informações de habilidades, competências e experiências para alimentar o modelo de IA.
Pré-condições	O Gestor deve estar autenticado no sistema com as devidas permissões.
Fluxo Principal	• O Gestor acessa a funcionalidade de gestão de colaboradores. • O Gestor seleciona a opção para adicionar um novo colaborador.

	• O sistema oferece duas opções: "Cadastro Manual" ou "Upload de Currículo". • Se "Upload de Currículo" for selecionado: 4.1. O Gestor envia o arquivo do currículo em PDF. 4.2. O sistema utiliza o modelo de IA (História #6) para ler e extrair as informações relevantes do arquivo. 4.3. O sistema preenche automaticamente os campos do formulário de cadastro com os dados extraídos. 4.4. O Gestor revisa os dados preenchidos e confirma o cadastro. • Se "Cadastro Manual" for selecionado: 5.1. O sistema exibe o formulário de cadastro de funcionários (História #1). 5.2. O Gestor preenche os campos com as informações do colaborador, como nome, cargo, habilidades, pontos fortes e fracos. 5.3. O Gestor salva as informações. • O sistema valida os dados e armazena as informações no Banco de Dados de Colaboradores. • O sistema exibe uma mensagem de confirmação.
Pós-condições	O novo colaborador está registrado no sistema e seus dados estão disponíveis para serem utilizados na geração de equipes.
Fluxo Alternativas (Exceções)	• Dados Inválidos: Caso algum campo obrigatório não seja preenchido, o sistema informa o Gestor e impede o salvamento até que os dados sejam corrigidos.

	Falha na Leitura do PDF: Se o sistema não conseguir extrair os dados do currículo, ele informa o Gestor e sugere o preenchimento manual.
--	--

ID	UC-01
Caso	Gerenciar colaboradores
Referência	"Formulário para cadastro de funcionários" e História #6: "Modelo de leitura de PDFs/currículos".
Atores	RH / Gestor de Equipes
Resumo	Este caso de uso permite que o Gestor cadastre novos colaboradores no sistema, seja preenchendo um formulário manualmente ou importando dados automaticamente a partir de um currículo em PDF. O objetivo é centralizar as informações de habilidades, competências e experiências para alimentar o modelo de IA.
Pré-condições	O Gestor deve estar autenticado no sistema com as devidas permissões.
Fluxo Principal	- O Gestor acessa a funcionalidade de gestão de colaboradores. - O Gestor seleciona a opção para adicionar um novo colaborador. - O sistema oferece duas opções: "Cadastro Manual" ou "Upload de Currículo". - Se "Upload de Currículo" for selecionado: 4.1. O Gestor envia o arquivo do currículo em PDF. 4.2. O

	sistema utiliza o modelo de IA (História #6) para ler e extrair as informações relevantes do arquivo. 4.3. O sistema preenche automaticamente os campos do formulário de cadastro com os dados extraídos. 4.4. O Gestor revisa os dados preenchidos e confirma o cadastro. • Se "Cadastro Manual" for selecionado: 5.1. O sistema exibe o formulário de cadastro de funcionários (História #1). 5.2. O Gestor preenche os campos com as informações do colaborador, como nome, cargo, habilidades, pontos fortes e fracos. 5.3. O Gestor salva as informações. • O sistema valida os dados e armazena as informações no Banco de Dados de Colaboradores. • O sistema exibe uma mensagem de confirmação.
Pós-condições	O novo colaborador está registrado no sistema e seus dados estão disponíveis para serem utilizados na geração de equipes.
Fluxo Alternativas (Exceções)	• Dados Inválidos: Caso algum campo obrigatório não seja preenchido, o sistema informa o Gestor e impede o salvamento até que os dados sejam corrigidos. • Falha na Leitura do PDF: Se o sistema não conseguir extrair os dados do currículo, ele informa o Gestor e sugere o preenchimento manual.
ID	UC-02

Caso	Gerar equipes com IA
Referência	Requisito Funcional "Geração automática de equipes balanceadas" e História #3: "Criação do modelo de IA para geração dos times".
Atores	RH / Gestor de Equipes
Resumo	Permite ao Gestor solicitar ao sistema a criação automática de equipes otimizadas para uma tarefa. O sistema utiliza um modelo de Inteligência Artificial para analisar os perfis dos colaboradores e sugerir a composição mais equilibrada, fornecendo um "coeficiente de aproveitamento" para cada sugestão.
Pré-condições	• O Gestor deve estar autenticado no sistema. • Deve existir um banco de dados com colaboradores e suas respectivas competências já cadastradas.
Fluxo Principal	• O Gestor seleciona a opção "Solicitação de Nova Equipe". • O Gestor define os parâmetros para a equipe (ex: objetivo do projeto, número de membros, habilidades essenciais). • O Gestor aciona a geração automática. • O sistema executa o "Processamento da IA", que analisa os dados dos colaboradores e busca a combinação ideal. • O sistema exibe ao Gestor uma ou mais sugestões de equipes.

	• Para cada sugestão, o sistema apresenta a lista de membros e o "Coeficiente de Aproveitamento" calculado. • O Gestor analisa a sugestão para tomar uma decisão (aceitar ou ajustar).
Pós-condições	Uma sugestão de equipe é apresentada ao Gestor, pronta para ser aprovada ou ajustada manualmente.
Fluxo Alternativas (Exceções)	• Colaboradores Insuficientes: Se a base de dados não possuir colaboradores com os perfis necessários para atender aos critérios, o sistema notifica o Gestor sobre a impossibilidade de formar uma equipe balanceada.

ID	UC-03
Caso	Ajustar equipe manualmente
Referência	História de Usuário #4: "Troca manual de membros".
Atores	RH / Gestor de Equipes
Resumo	Este caso de uso permite que o Gestor modifique a composição de uma equipe sugerida pela IA. O objetivo é considerar fatores humanos e subjetivos que o modelo pode não capturar, permitindo a troca, adição ou remoção de membros. Cada ajuste manual serve como feedback para o aprimoramento contínuo da IA.

Pré-condições	Uma equipe foi gerada pela IA e está sendo exibida para o Gestor.
Fluxo Principal	• Após visualizar a sugestão da IA, o Gestor decide realizar um ajuste e seleciona a opção "Ajustar Manualmente". • O sistema abre a interface de troca de membros. • O Gestor seleciona um membro da equipe para ser substituído. • O sistema exibe uma lista de colaboradores disponíveis e destaca opções que maximizam o "Coeficiente de Aproveitamento". • O Gestor seleciona um novo colaborador para integrar a equipe. • O sistema atualiza a composição da equipe. • O Gestor confirma a alteração. • O sistema salva a nova "Equipe Final". • O sistema registra a alteração como um "Feedback para o Modelo" de IA.
Pós-condições	A equipe é salva com a nova formação definida pelo Gestor, e a IA recebe dados para aprimorar futuras sugestões.
Fluxo Alternativas (Exceções)	• Cancelar Ajuste: O Gestor pode, a qualquer momento, cancelar a operação e reverter para a sugestão original da IA.

1.5.2. Histórias de usuário

As histórias de usuário apresentadas nesta seção fornecem uma visão objetiva das necessidades e expectativas dos diferentes perfis de usuários em relação ao sistema. Estruturadas a partir da perspectiva do próprio usuário, elas visam capturar o valor desejado, facilitando a compreensão do propósito de cada funcionalidade e orientando o desenvolvimento de forma centrada na experiência do usuário.

Neste estágio do projeto, as histórias de usuário selecionadas representam o conjunto inicial de requisitos priorizados. Novas histórias serão adicionadas e aprimoradas de maneira iterativa, acompanhando o avanço do desenvolvimento e permitindo que o sistema evolua continuamente conforme surgem novas demandas ou necessidades de refinamento.

ID	RIA-01
Título	Formulário para cadastro de funcionários
História	Como administrador da empresa, quero registrar meus funcionários por meio de um formulário no RANKIA, para que eu possa gerenciar seus cadastros e acessar funcionalidades que facilitem o acompanhamento das atividades.
Prioridade	Urgente
Estimativa	2,5 semanas
Critério de Avaliação	Formulário completo com opções de filtros e com campos para envio de arquivos

ID	RIA-02
Título	Modelo de leitura de PDFs/Currículos
História	Como analista de seleção, quero implementar um modelo capaz de tratar os dados dos currículos, para que seja possível aplicar corretamente os pesos utilizados pelo seletor de qualidade.
Prioridade	Alta
Estimativa	4 semanas
Critério de Avaliação	O modelo deve estar funcional e processar os dados conforme os pesos definidos. Deve possuir pelo menos 5 testes realizados comprovando seu funcionamento. A solução só será aceita após a validação e aprovação dos pesos utilizados no processo.

ID	RIA-03
Título	Treinamento de IA
História	Como responsável pela análise de equipes, quero treinar a IA de balanceamento utilizando o banco atual de currículos de teste, para que o sistema consiga formar equipes equilibradas e com melhor desempenho.
Prioridade	Alta
Estimativa	2 semanas
Critério de Avaliação	A média de qualidade das equipes geradas pela IA deve

	ser superior a 79%.
--	---------------------

ID	RIA-04
Título	Criação de medidor de qualidade de equipe
História	Como gestor de equipes, quero utilizar o coeficiente atual de qualidade das equipes para gerar um indicador de qualidade no perfil de cada equipe, permitindo uma visualização clara e objetiva do seu desempenho.
Prioridade	Média
Estimativa	5 semanas
Critério de Avaliação	O indicador deve estar funcional e disponível para acesso fácil e imediato no perfil da equipe.

ID	RIA-05
Título	Troca manual de membros
História	Como usuário, gostaria de poder alterar manualmente alguns membros da equipe caso a seleção automática não seja totalmente adequada, garantindo maior flexibilidade na composição.
Prioridade	Baixa
Estimativa	3 Dias
Critério de Avaliação	A interface deve ser intuitiva e amigável, permitindo ajustes manuais nos membros da equipe.

	O sistema deve apresentar sugestões de possíveis trocas, auxiliando o processo de escolha.
--	--

ID	RIA-06
Título	Geração de relatórios
História	Como usuário, ao gerar a equipe, gostaria de receber por algum meio de comunicação a composição final das equipes, para que eu possa visualizar rapidamente os membros selecionados.
Prioridade	Baixa
Estimativa	1 Semana
Critério de Avaliação	O sistema deve enviar automaticamente uma visualização clara e completa, contendo as características de cada membro da equipe gerada.

ID	RIA-07
Título	Acompanhamento de desenvolvimento
História	Como funcionário da empresa, gostaria de acompanhar e visualizar feedbacks sobre minhas competências, entendendo quais habilidades preciso aprimorar e quais já estão bem avaliadas, para orientar meu desenvolvimento profissional.
Prioridade	Média
Estimativa	3 Semanas

Critério de Avaliação	O funcionário deve conseguir acessar essas informações por meio de uma interface intuitiva, que apresente uma visualização clara dos dados, destacando suas habilidades atuais e indicando pontos de melhoria.
------------------------------	--

ID	RIA-08
Título	Recomendações de treinamentos
História	Como funcionário, quero receber recomendações de cursos, bootcamps, treinamentos e materiais relacionados às minhas deficiências e áreas de melhoria, para que eu possa me desenvolver profissionalmente de forma mais eficiente.
Prioridade	Média
Estimativa	4 Semanas
Critério de Avaliação	A IA/sistema deve indicar no mínimo um curso, treinamento ou material adequado às habilidades que o funcionário precisa aprimorar, com base em sua avaliação atual.

2. Modelagem dos requisitos

2.1. Introdução

Após realizarmos a extração dos requisitos, torna-se necessário estruturar e organizar essas informações por meio da modelagem. Essa etapa permite compreender melhor as ideias apresentadas no tópico 1, as transformando em

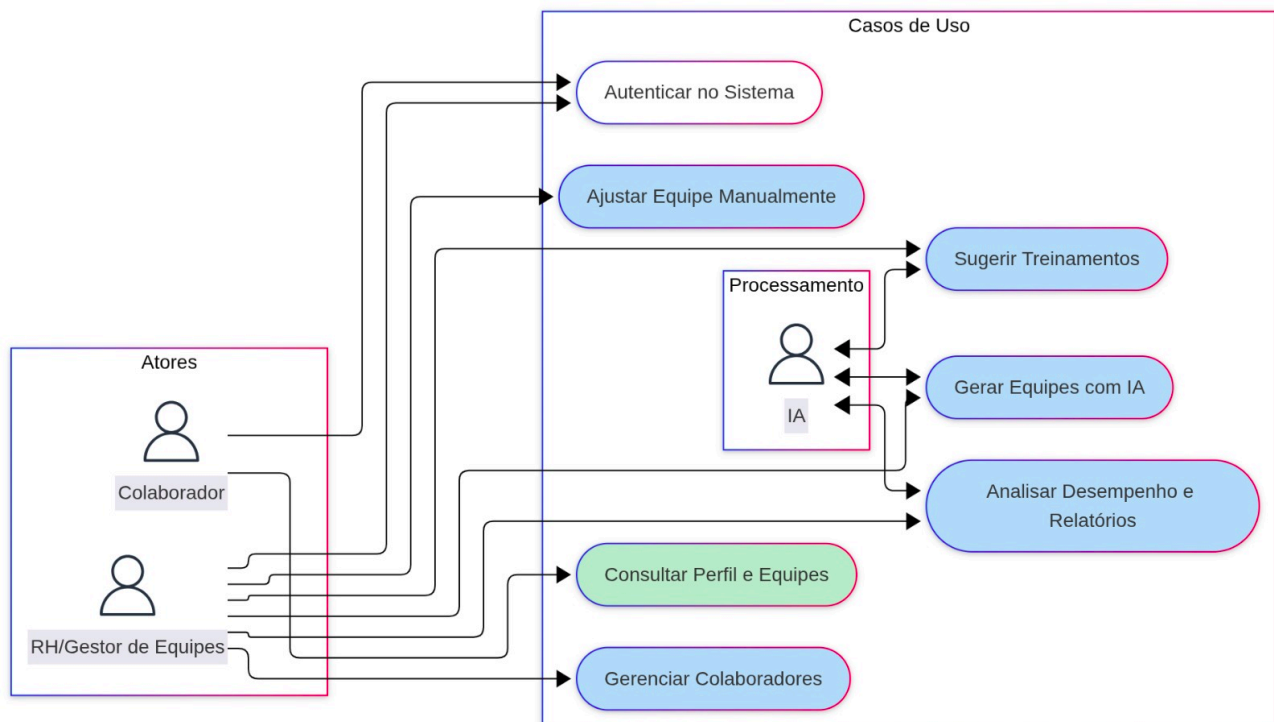
representações visuais e dinâmicas que facilitam o entendimento do sistema como um todo.

Nesta seção, apresentaremos os diagramas ensinados em sala de aula e utilizados no projeto: Diagrama de Classes, Diagrama de Caso de Uso, Diagrama de Sequência e Diagrama de Atividade.

2.2. Diagrama de Caso de Uso

O primeiro modelo de diagrama que realizamos, e porventura também foi o primeiro a aprendermos em classe, é o Diagrama de Caso de Uso UML, super importante para a base do projeto. Logo após realizarmos as extrações de requisitos, ponderação das regras de negócio, decisão de métricas e principalmente a formulação da ideia do projeto, realizamos as Histórias de Usuários e Casos de Uso. Para melhor visualização e compreensão das interações entre os usuários (atores) e o sistema (RANK-IA), utilizamos o diagrama.

2.2.1. Diagrama 1



Fonte: Elaboração Própria

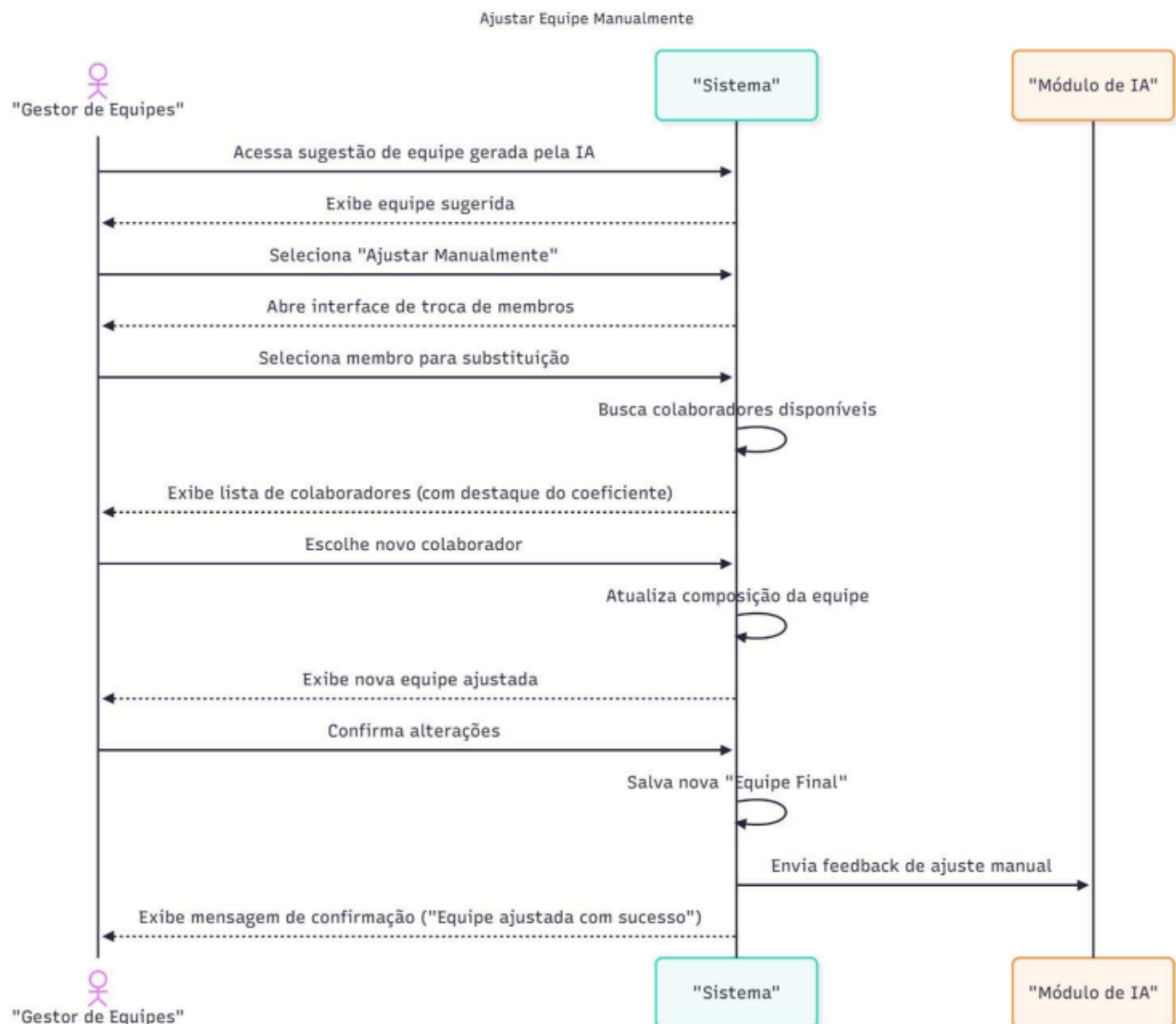
2.2.2. Descrição do diagrama 1

Casos de uso	Breve descrição	Atores principais	Fluxo de sucesso
Autenticar no sistema	Permite que colaboradores e gestores acessem o sistema com credenciais válidas.	Colaborador / RH / Gestor	É informado o login e senha, ambos são validados e o acesso é concedido.
Sugerir treinamentos	O sistema recomenda treinamentos com base em desempenho e métricas.	Gestor / IA	O sistema/Gestor analisa os dados do funcionário e anexa treinamentos para o mesmo.
Gerar equipes	O sistema forma equipes automaticamente usando um algoritmo de IA.	IA / Gestor	O gestor solicita e a IA processa e retorna equipes sugeridas.
Analisar desempenho e relatórios	Permite visualizar métricas, análises e relatórios consolidados.	IA / Gestor	O Gestor acessa relatórios fornecidos pelo sistema.
Gerenciar colaboradores	Permite ao RH adicionar, editar ou remover colaboradores.	RH / Gestor	O gestor acessa o menu de gerenciamento e executa a ação desejada.
Ajustar equipe manualmente	Permite ao gestor modificar equipes manualmente.	RH / Gestor	O gestor acessa a área de equipes, altera membros e salva as mudanças.
Cadastrar Usuário	O gestor pode cadastrar um novo funcionário no sistema.	RH / Gestor	O gestor acessa a área de sign up e adiciona um novo funcionário ao plantel.
Consultar Perfil e Equipes	Permite visualizar informações de perfil e composição de equipes.	RH / Gestor	O usuário/Gestor acessa a página de consulta e visualiza seus dados e equipes.

2.3. Diagrama de Sequência

Os diagramas de sequência da UML podem ser usados para modelagem comportamental. Uma vez que os eventos tenham sido identificados pelo exame de um caso de uso, é possível fazer sua modelagem em um diagrama de sequência, representando os eventos e como eles provocam o fluxo de interação de um objeto para outro (ou outros) em função do tempo.

2.3.1. Visualização do Diagrama de Sequência



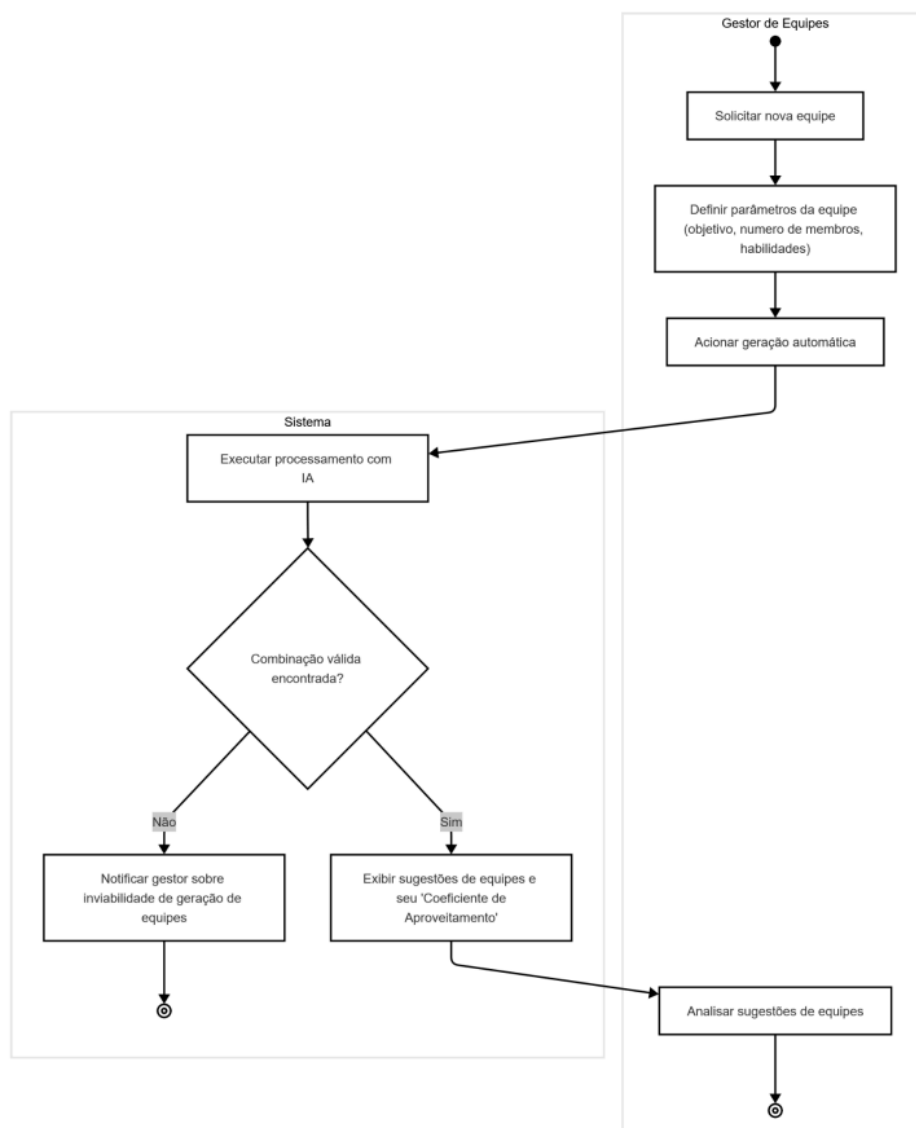
Fonte: Elaboração Própria

2.4. Diagrama de Atividade

Os Diagramas de Atividade têm como propósito representar detalhes do processamento, enfatizando o fluxo de controle de uma atividade para outra, independentemente do nível de abstração procedural. Realizando a análise de cenários específicos, o Diagrama de Atividades da UML complementa o caso de uso por meio de uma representação gráfica do fluxo de interação.

No nosso projeto, criamos dois Diagramas de Atividade, cada um contemplando um cenário do projeto.

2.4.1. Visualização do Diagrama de Atividades



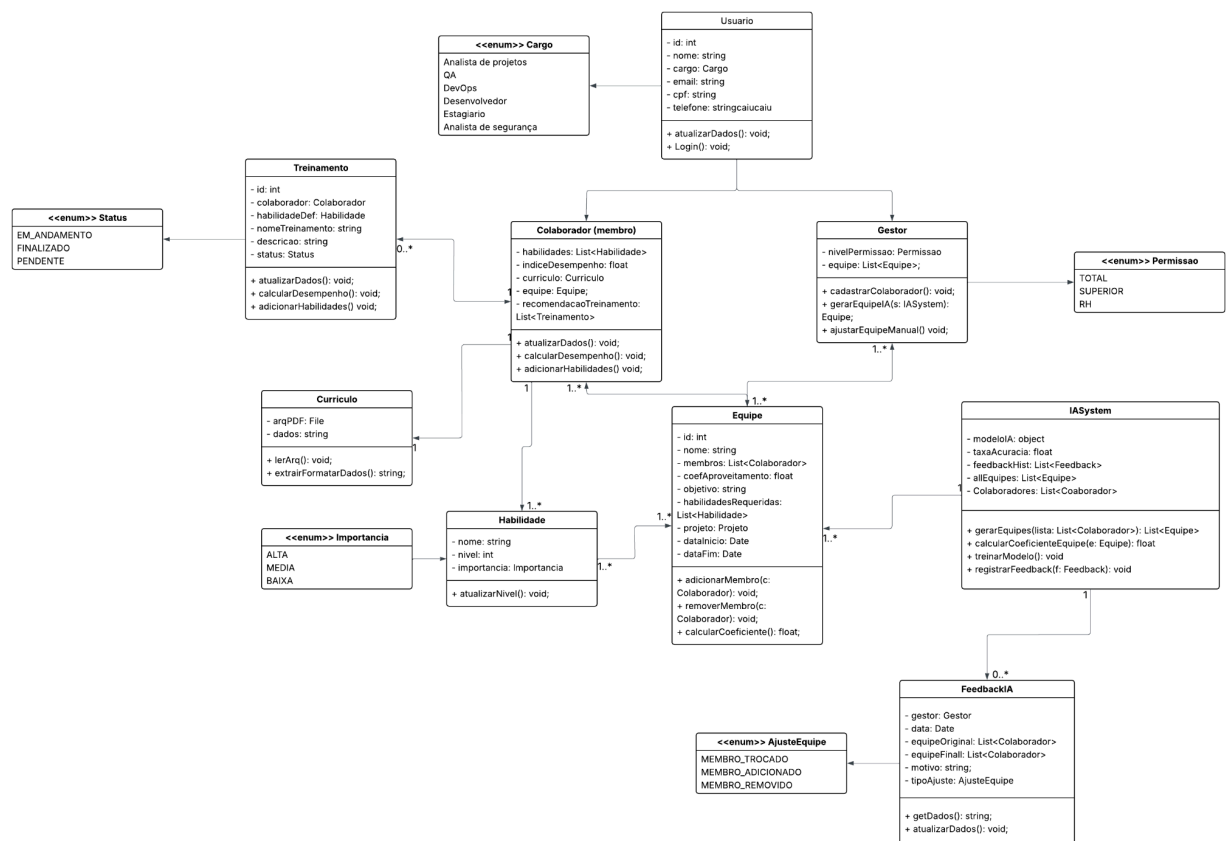
Fonte: Elaboração Própria

2.5. Diagrama de Classes

O diagrama de classes UML é essencial para visualizar antecipadamente a estrutura do software, ajudando a organizar as classes, seus atributos, métodos e relações antes da implementação. Ele torna o sistema mais claro e coeso, além de permitir ajustes sempre que surgirem mudanças no projeto.

No caso do RANK-IA, o diagrama criado não representa todas as funcionalidades, e sim uma visão conceitual das partes mais importantes do sistema. Mesmo reduzido, ele reúne os elementos essenciais, como herança, ENUMs e associações, garantindo uma compreensão geral da arquitetura do aplicativo.

2.5.1. Visualização do Diagrama de Classes



Fonte: Elaboração Própria

3. Processo de desenvolvimento de software

3.1. Introdução

3.1.1. Propósito

De primeira instância, foi pensado pela equipe que, diante da necessidade de organizar as tarefas e estabelecer uma linha clara de desenvolvimento do software, seria necessário adotar uma metodologia ágil. Considerando as dificuldades de temporização para reuniões e as diferenças de horários entre os membros, optou-se por utilizar o Scrum como base do processo de desenvolvimento, realizando uma mesclagem com o Kanban para proporcionar uma melhor visualização do fluxo de trabalho, tendo em vista sua abordagem por quadro.

3.1.2. Objetivos

O principal objetivo do processo é garantir organização, colaboração e entregas contínuas ao longo do desenvolvimento do projeto. Com o uso combinado do Scrum e do Kanban, buscamos:

- Melhorar a comunicação entre os integrantes;
- Acompanhar o progresso por meio de sprints e quadros visuais;
- Garantir que as entregas ocorram dentro dos prazos definidos;
- Adaptar o planejamento conforme o feedback das iterações;
- Facilitar a visualização das tarefas em andamento, concluídas e pendentes.

3.1.3. Justificativa do Modelo

A escolha do Scrum se deve à sua flexibilidade e foco em resultados incrementais, o que permite adaptar o projeto às mudanças de escopo e à

disponibilidade dos participantes.

A utilização do Kanban como apoio traz benefícios complementares, como a visibilidade do fluxo de trabalho e a gestão de prioridades em tempo real.

Essa combinação possibilita à equipe equilibrar planejamento estruturado (por meio das sprints do Scrum) com agilidade visual e contínua (por meio do quadro Kanban), estimulando o trabalho em equipe, a auto-organização e a entrega constante de valor ao cliente ou usuário final.

3.2. Estruturação do processo Scrumban

O processo será dividido como qualquer outro processo desenvolvido puramente com Scrum, portanto, o processo será dividido em ciclos curtos de desenvolvimento denominados **sprints**, cada um com duração aproximada de **duas semanas**. Cada Sprint é composta por histórias de usuário que são blocos que englobam tarefas requisitadas pelo usuário ou stakeholder.

A cada sprint, novas funcionalidades serão planejadas, desenvolvidas e testadas com o intuito de melhorar e completar o software de maneira cíclica e confiável.

3.2.1. Ferramenta de desenvolvimento

No mercado de desenvolvimento de software existem diversas ferramentas que permitem a aplicação das metodologias ágeis estudadas na matéria. Após analisarmos com atenção algumas das mais conhecidas, como Trello, ClickUp, Notion, Jira, entre outras amplamente utilizadas na indústria de software, selecionamos o [Taiga.io](https://taiga.io).

A escolha se deu pelo fato de o Taiga.io oferecer suporte à criação e utilização do processo Scrumban de forma flexível e gratuita, o problema que enfrentamos ao utilizar a ferramenta foi em relação aos indicadores de histórias de usuário e tasks que é sequencial e fixo, ou seja, o usuário não tem controle sobre. Embora não seja a melhor ferramenta disponível no mercado, suas limitações acabam sendo compensadas por suas qualidades e pela facilidade

de uso no contexto do nosso projeto.

3.2.2. Definição das colunas do quadro

- Novo: Tarefas recém-criadas que ainda não foram iniciadas. É o ponto inicial do fluxo.
- Em Andamento: Tarefas que estão sendo desenvolvidas pela equipe no momento. Representam o progresso ativo da Sprint.
- Pronto para teste: Reúne tarefas cuja implementação já foi concluída e que aguardam revisão, validação ou testes.
- Fechado: Tarefas completamente finalizadas e validadas. Indica que o trabalho foi entregue, aceito e discutido em uma daily.
- Precisa de informação: Contém tarefas que não podem avançar por falta de dados, esclarecimentos ou recursos, evitando que impedimentos passem despercebidos.

3.2.3. Ciclo de desenvolvimento

3.2.3.1. Duração

No projeto como um todo, existe uma data de entrega máxima e, de acordo com todas as funcionalidades que devem ser aplicadas, extraídas por meio das user stories, é feita a relação das sprints com suas durações variadas.

3.2.3.2. Incrementos entregáveis

A cada sprint, o time gera um incremento do produto, que corresponde a uma parte funcional e validada do software. Esses incrementos devem estar completamente desenvolvidos, testados e prontos para uso, ainda que representem apenas uma fração do sistema final. O objetivo é garantir que, ao término de cada ciclo, exista algo concretamente utilizável e que agregue valor ao usuário ou stakeholder, permitindo que o projeto evolua de forma contínua e verificável.

3.2.3.3. Reavaliações contínuas

Durante todo o processo de desenvolvimento, o progresso é constantemente revisado por meio de reuniões rápidas, análises do fluxo de trabalho e feedbacks obtidos a partir dos incrementos entregues. Essas avaliações têm como finalidade identificar possíveis melhorias, ajustar prioridades e corrigir desvios do planejamento original. Esse processo contínuo garante que o projeto mantenha alinhamento com as necessidades do usuário e com os objetivos definidos pela equipe.

3.2.4. Papéis e Responsabilidades

3.2.4.1. Scrum Master

Na RANK-IA contamos com apenas um Scrum Master, o Murilo Gonzaga, responsável por garantir que o processo Scrum seja aplicado corretamente. Esse papel inclui remover impedimentos, facilitar reuniões, apoiar a equipe no entendimento das práticas ágeis e assegurar que o fluxo de trabalho permaneça eficiente e transparente para todos os membros.

3.2.4.2. Product Owner

O responsável com maior autoridade dentro do projeto é o Fernando Hiroshi, cujo objetivo é garantir que o produto entregue atenda às necessidades dos usuários e aos requisitos definidos. Ele prioriza as tarefas, organiza o backlog e assegura que a equipe esteja sempre alinhada ao propósito do projeto.

3.2.4.3. Developers

A equipe de desenvolvimento é composta por Matheus Magro e Igor dos Reis, responsáveis por implementar as funcionalidades, corrigir bugs, realizar testes e garantir que o produto evolua de acordo com o planejado.

3.3. Planejamento das Sprints

A organização do processo de desenvolvimento teve início na etapa de

levantamento e aquisição de requisitos. Em seguida, foi realizada a extração das informações dos casos de uso, histórias de usuário, *tasks* e modelagens, para a criação das histórias e tarefas dentro do [Taiga.io](https://taiga.io).

Essa etapa, realizada pelo Product Owner, tem como objetivo alimentar o *product backlog* com todas as informações necessárias para o desenvolvimento.

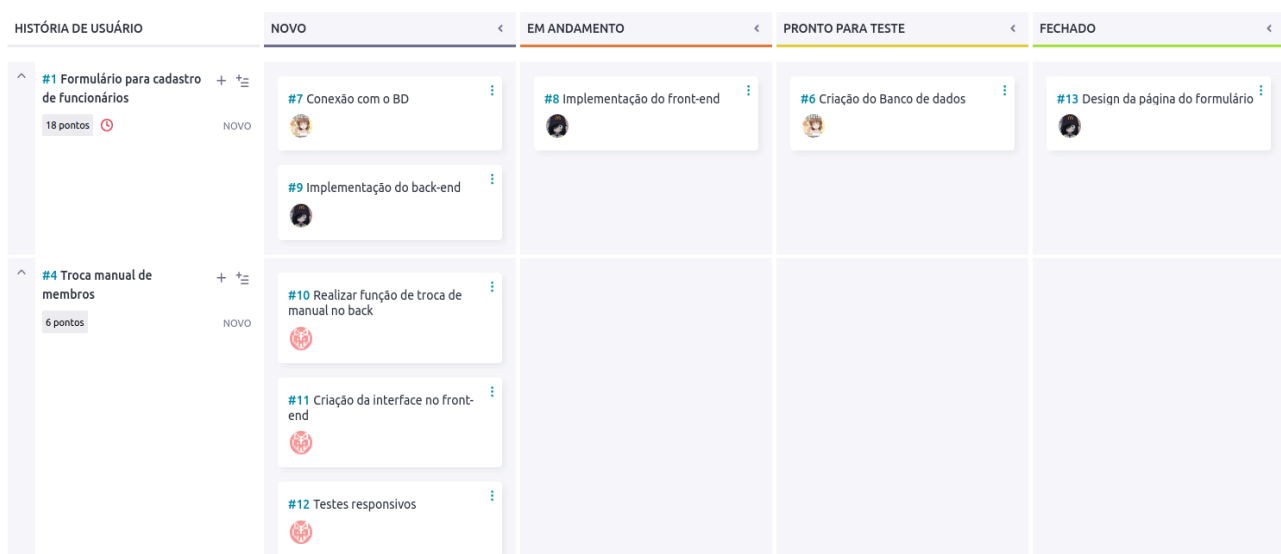
3.3.1. Sprint 1

Nossa primeira sprint foi montada em sala de aula durante as atividades propostas pelo professor André. Nessa sprint, o Product Owner analisou as histórias de usuário e os casos de uso que já havíamos desenvolvido. A partir dessa análise, foram escolhidas as histórias de usuário:

- 1i – Troca manual de membros
- 5 – Formulário para cadastro de usuários.

A escolha dessas histórias se deve ao fato de serem funcionalidades primitivas e fundamentais do produto. Elas precisam ser implementadas antes das demais, pois servem de base para futuras funcionalidades que dependem dessas estruturas iniciais.

3.3.1.1. Visualização do quadro kanban



Fonte: Elaboração Própria (TAIGA.IO)

Ao observarmos o quadro na seção 3.3.1.1, percebemos que os identificadores das histórias de usuário não estão iguais aos utilizados nas atividades realizadas em classe. Isso acontece porque não temos controle total sobre os IDs gerados automaticamente pela plataforma Taiga, o que causa pequenas diferenças na numeração exibida.

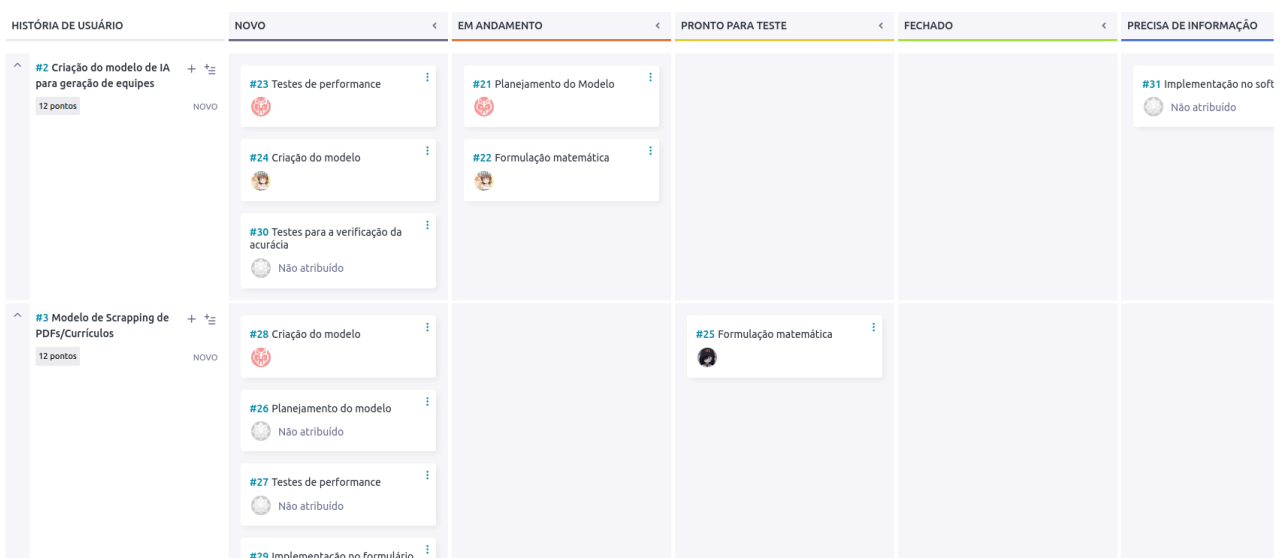
3.3.2. Sprint 2

Na segunda Sprint do projeto, o foco principal foi a parte matemática e o core da aplicação. Como a etapa de extração de dados já estaria encaminhada após a Sprint 1, surgiram duas questões importantes: como essas informações armazenadas no BD seriam utilizadas e como seria o modelo de IA responsável pela funcionalidade central do sistema, sua base matemática, seu funcionamento e suas capacidades.

Diante disso, o Scrum Master realizou uma nova análise e adicionou duas histórias de usuário para serem desenvolvidas nesta sprint:

- 3 – Criação do modelo de IA para geração dos times e
- 6 – Modelo de leitura de PDFs/Currículos.

3.3.2.1. Visualização do quadro Kanban



Fonte: Elaboração Própria (TAIGA.IO)

Ao observarmos o quadro na seção 3.3.2.1, percebemos que os identificadores das histórias de usuário não estão iguais aos utilizados nas atividades realizadas em classe. Isso acontece porque não temos controle total sobre os IDs gerados automaticamente pela plataforma Taiga, o que causa pequenas diferenças na numeração exibida.

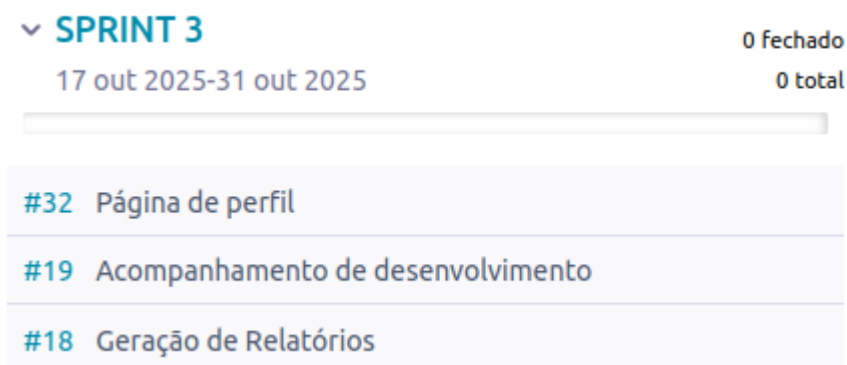
3.3.3. Sprint 3

A terceira Sprint não foi projetada inicialmente, mas fizemos uma estimativa do que poderia ser desenvolvido caso todas as tasks das sprints anteriores tivessem sido concluídas dentro do prazo. Dessa forma, criamos uma sprint imaginária com histórias de usuário mais simples e com implementações importantes, porém de prioridade secundária quando comparadas às das outras sprints. As histórias selecionadas foram:

- 32 – Página de perfil
- 19 – Acompanhamento de desenvolvimento
- 18 – Geração de relatórios.

A ideia dessa sprint, com duração aproximada de duas semanas e meia, é que as histórias de usuário possuam uma relação entre si, promovendo comunicação constante entre os membros da equipe, mesmo quando estiverem trabalhando em *tasks* de histórias diferentes.

3.3.3.1. Projeção da sprint



Fonte: Elaboração Própria (TAIGA.IO)

3.4. Execução e Acompanhamento

3.4.1. Daily meetings

Todas essas atividades precisam de atualizações diárias para que o Scrum Master tenha uma noção clara do que está ocorrendo durante a sprint. Com essa visão geral do dia a dia, tornam-se mais evidentes os pontos de melhoria, possíveis impedimentos e necessidades da equipe, permitindo ajustes rápidos para que a linha de produção se mantenha eficiente.

3.4.2. Retrospectivas

Ao final de cada sprint é realizada uma reunião de retrospectiva da equipe, onde cada membro pontua o que foi feito de maneira eficiente e o que deixou a desejar. O objetivo é evitar que os mesmos erros se repitam nas próximas sprints, otimizando tanto o trabalho quanto às relações entre os integrantes.

3.4.3. Story Points

No projeto RANK-IA, os *story points* são utilizados para estimar o esforço necessário para concluir cada história, considerando fatores como complexidade, tamanho, possíveis impedimentos e o nível de conhecimento exigido. Essa abordagem ajuda a equipe a planejar melhor as sprints e a entender o peso real de cada tarefa.

Além disso, no Taiga os story points também funcionam como um incentivo para os desenvolvedores, já que são atribuídos individualmente e servem como uma métrica simples de produtividade. No projeto, esses pontos foram distribuídos por diferentes setores, permitindo visualizar de forma mais clara onde está concentrado o trabalho de cada área.

3.4.3.1. Story points no Taiga

PONTOS	
UX	0
Design	0
Front	2
Back	10
total de pontos	12

Fonte: Elaboração Própria (TAIGA.IO)

4. Qualidade e métricas de software

4.1. Introdução

A qualidade de software é um conceito multifacetado que vai além da simples ausência de erros. Conforme definido por Pressman e adotado na disciplina, a qualidade é "um produto útil que forneça valor mensurável para aqueles que o produzem e para aqueles que o utilizam".

Para o projeto RankIA, a abordagem de qualidade baseia-se na norma ISO/IEC 25010, focando tanto na *Qualidade do Produto* (características estáticas e dinâmicas do sistema) quanto na *Qualidade em Uso* (interação do usuário no contexto real). Além disso, seguindo os conceitos fundamentais de medição de software, distinguimos:

4.2. Objetivos da qualidade

O objetivo da equipe é garantir que o RankIA atenda às características do modelo de qualidade de produto definido na norma. As prioridades para o sistema são:

1. Adequação Funcional: O sistema deve fornecer correta e completamente a formação de equipes via IA, atendendo à precisão esperada pelos gestores.
2. Confiabilidade: O sistema deve manter-se disponível e tolerante a falhas, garantindo a maturidade dos dados de competências dos colaboradores.
3. Usabilidade: Foco na operabilidade e proteção contra erros de usuários (RH e Gestores), garantindo uma interface intuitiva.
4. Facilidade de Manutenção: O código deve apresentar alta modularidade e testabilidade para permitir a evolução dos algoritmos de IA.

4.3. Qualidade de Uso

A qualidade de uso refere-se ao impacto e percepção que um software tem

sobre o usuário do sistema. Para o RANK-IA, a qualidade de uso irá ter impacto diretamente na experiência dos funcionários de RH e gestores de empresas para criação de equipes melhores e eficientes. Para avaliação da qualidade, as seguintes características serão priorizadas para o projeto:

- **Eficácia:** trata-se de medir a precisão e completude com a qual os gestores atingem suas metas. No contexto do RANK-IA, isso se traduz na capacidade da IA de sugerir uma equipe que realmente possua as competências necessárias e requisitadas, sem exigir que o gestor descarte a sugestão inteira e iniciar novamente do zero.
- **Eficiência:** trata-se de verificar se a relação entre o desempenho e recursos com os resultados produzidos. O sistema deve reduzir drasticamente o tempo que um gestor levaria para cruzar dados de planilhas e currículos manualmente. Caso o gestor demore a mesma quantidade de tempo para realizar a configuração do software para conseguir formar uma equipe do que montando uma equipe manualmente, o software falha nesse quesito.
- **Satisfação:** trata-se do conforto e satisfação do usuário com o sistema. Para o RANK-IA satisfazer esse critério de satisfação, a interface deve ser bem agradável, de fácil uso e com explicações do porquê uma determinada equipe foi formada.
- **Ausência de Riscos:** trata-se de focar na minimização de riscos econômicos e de segurança. Para isso, o RANK-IA deve garantir que a formação de equipes não gere viés discriminatório ou exponha dados sensíveis dos colaboradores (LGPD).

4.4. Qualidade de Produto

4.4.1. Desempenho

Definição: Para atingir de forma completa os objetivos dos usuários, o

sistema RANK-IA deve ter alto desempenho. Em termos de qualidade, isso se traduz na velocidade entre a ação do usuário de solicitar a formação das equipes de um time e a resposta do software em retornar as equipes formadas de acordo com a IA de geração de times. Dessa forma podemos manter o padrão de qualidade esperados na assinatura de um sistema feito diretamente para o conforto do gestor.

Objetivo: A porcentagem de equipes formadas dentro do tempo adequado deve ser maior que 90% para todos os tamanhos de times.

4.4.2. Escalabilidade do software

Definição: A capacidade do sistema de crescer e lidar com um aumento na carga de trabalho, especificamente o número de usuários ativos simultaneamente. A importância da alta escalabilidade visa uma boa consistência e preparo para possíveis aumentos na quantidade de usuários simultâneos.

Objetivo: O tempo médio mínimo aceitável de resposta no site é 2 segundos. A taxa de falhas deve ser abaixo de 99%.

4.4.3. Usabilidade

Definição: A capacidade com que o usuário pode utilizar, aprender e obter resultados com o software. A usabilidade tem como principal característica garantir que o software seja intuitivo, eficiente e fácil de usar. Para isso, algumas das features presentes no RANK-IA são a interface de "arrastar e soltar" para ajustes manuais de equipes e a visualização gráfica de radares de competências, que traduzem dados complexos da IA em informações visuais rápidas.]

Objetivo:

- Atingir uma pontuação acima de 70 na escala SUS (System Usability Scale) nos testes com usuários.
- Garantir que um gestor consiga realizar o primeiro ajuste de equipe em

menos de 3 cliques.

4.4.4. Segurança

Definição: A capacidade do software de proteger dados e informações contra o uso, alteração e destruição não autorizados. Para isso, os pilares principais seguidos trata-se da confiabilidade, apenas usuários têm acessos aos próprios dados, integridade, dados não podem ser alterados por usuários não autorizados, e disponibilidade, os dados estão facilmente disponíveis aos usuários quando forem requisitados. No sistema RANK-IA essa segurança é essencial para que os usuários do sistema possam ter pleno conforto de que o aplicativo para a resolução de seus problemas é altamente seguro e criptografado.

Objetivo: 0 (zero) vulnerabilidades de severidade alta ou crítica encontradas no teste de penetração. Todas as senhas devem usar criptografia forte (ex: bcrypt).

4.4.5. Compatibilidade

Definição: A capacidade do software de interagir e coexistir com outros sistemas, trocando informações de forma eficaz. No contexto do RankIA, isso é vital para a importação de dados (currículos) e exportação de resultados. O sistema deve ser capaz de processar documentos em formatos padrão de mercado (PDF, DOCX) sem perda de formatação e exportar os relatórios de equipes em formatos universais (CSV, XLSX) para integração com outros sistemas de RH.

Objetivo:

- Taxa de sucesso de 99% na leitura de arquivos PDF gerados por diferentes versões do Adobe Acrobat e Microsoft Word.
- Renderização correta da interface (sem quebra de layout) nos três principais navegadores do mercado: Chrome, Firefox e Edge.

4.4.6. Confiabilidade

Definição: A capacidade do software de desempenhar suas funções de forma consistente e correta sob condições definidas. Para o RankIA, a confiabilidade é crítica na persistência dos dados das equipes formadas. O sistema deve garantir que, uma vez que uma equipe é salva ou um ajuste manual é feito, essa informação não seja perdida mesmo em caso de reinicialização do servidor. Além disso, a IA deve apresentar comportamento determinístico para as mesmas entradas (inputs idênticos devem gerar sugestões idênticas se o modelo não tiver sido retreinado).

Objetivo:

- Disponibilidade do serviço (Uptime) de 99,5% durante o horário comercial (8h às 18h).
- MTBF (Mean Time Between Failures) superior a 720 horas de operação contínua

4.4.7. Portabilidade

Definição: A facilidade com que o software pode ser transferido de um ambiente para outro ou acessado em diferentes dispositivos. Sendo uma aplicação Web, a portabilidade do RankIA foca na responsividade e na independência de hardware do cliente. O sistema deve se adaptar automaticamente a diferentes resoluções de tela, permitindo que gestores utilizem tablets ou notebooks com a mesma eficácia que desktops de alta resolução.

Objetivo:

- Aprovação em 100% dos testes de responsividade para resoluções a partir de 1024px (tablets em modo paisagem e desktops).
- O sistema não deve exigir instalação de plugins ou extensões proprietárias no navegador do cliente.

4.5. Métricas

Para garantir que o RANK-IA atenda aos objetivos de eficiência e qualidade propostos, é necessário estabelecer um processo de medição contínuo. Para isso, serão necessárias medidas (coleta de dados) e as métricas (que relacionam essas medidas para fornecimento de indicadores). O objetivo central destas métricas é fornecer aos desenvolvedores e aos stakeholders (Gestores e RH) indicadores claros sobre a precisão do algoritmo de IA e a manutenibilidade do código, permitindo que a equipe tome decisões baseadas em dados, e não apenas em suposições.

4.5.1. Métricas de Uso

As métricas de qualidade de uso concentram-se em transformar a percepção e o desempenho do Gestor de RH em dados quantificáveis, garantindo que a automação traga ganho real de eficiência. Baseando-se nos atributos de qualidade de projeto e controle de qualidade, definimos:

- **Complexidade da Interface:** contabiliza o número médio de cliques para chegar a uma função ou conteúdo típico. No RANK-IA, mediremos quantos cliques são necessários desde o login até a geração da primeira equipe, visando minimizar o esforço de navegação.
- **Tempo de Conclusão:** mede a diferença entre o tempo estimado que o usuário leva para completar uma tarefa e o tempo que o usuário realmente aguarda em tela.
- **Densidade de Erros de Operação:** baseada na contagem de número de erros encontrados, esta métrica quantifica quantas vezes o usuário comete erros de validação durante o uso. Uma densidade alta indica que a interface não é intuitiva o suficiente.

4.5.2. Métricas de Qualidade

Focadas na equipe de desenvolvimento, essas métricas utilizam análise estática do código, principalmente orientada a objetos, para garantir a manutenibilidade e controlar a complexidade do sistema.

- Complexidade Ciclométrica: métrica que mede a complexidade de controle (quantidade de caminhos independentes) em uma função ou método de um programa, indicando a facilidade de compreensão e testabilidade do código.
- Acoplamento entre Objetos (CBO): métrica que mede o número de chamadas de uma classe para outra, servindo como indicador de manutenção, pois quanto maior o acoplamento, mais difícil é manter o sistema.
- Falta de Coesão em Métodos (LCOM): métrica que mede o quanto os métodos de uma classe acessam atributos em comum. Valores baixos são buscados, pois indicam alta coesão e melhor organização interna da classe.